RAJSHAHI UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

LAB REPORT

-

"DATA MINING"

Submitted by

Md Naim Parvez
Roll No: 2003015
Section: A
4th Year (Odd Semester)

Submitted to

A.F.M. Minhazur Rahman
Assistant Professor
Department of CSE
Rajshahi University of Engineering & Technology

Course Code: 4120

Course Title: Data Mining Sessional

Contents

List of Figures

1 Introduction

1.1 Motivation and Dataset Selection

Road safety is a critical global concern, with traffic accidents ranking among the leading causes of injury and death worldwide. Identifying and predicting high-risk road conditions are essential steps toward designing smarter transportation systems and saving lives.

For this project, the **Road Accident Risk Dataset** was selected. This synthetic dataset simulates real-world road conditions and represents a diverse range of environmental, infrastructural, and temporal variables. Its design supports predictive modeling tasks focused on estimating accident probabilities across different road segments.

Why Road Accident Risk Dataset?

- *Real-World Context:* The dataset reflects the multidimensional nature of traffic safety—weather, lighting, surface type, and traffic density all interact to determine risk.
- *Classification Feasibility:* By categorizing segments into low-, medium-, and high-risk classes, supervised learning algorithms can be effectively applied.
- *Clustering Potential:* Numerical and categorical features allow for unsupervised exploration to uncover natural accident patterns.
- *Data Challenge:* The dataset includes missing values and simulated anomalies, providing an opportunity to develop robust preprocessing pipelines.

The dataset was sourced from an active Kaggle competition, providing an opportunity to benchmark solutions against the data science community. The dataset can be accessed at: https://www.kaggle.com/competitions/playground-series-s5e10/data.

1.2 Lab Objectives and Task Overview

The work follows a multi-stage data mining pipeline, composing tasks that transform raw data into actionable insights:

1. *Lab 1 – Dataset Analysis:*Understanding data structure, dimensions, and classification problem formulation. This initial step is crucial for identifying data types, detecting potential issues early, and determining the appropriate machine learning approach before investing time in modeling
2. *Lab 2 – Data Preprocessing:* Handling missing values, outliers, encoding, scaling, and similarity measures. Preprocessing is essential because algorithms cannot handle missing data or categorical variables directly, and unscaled features can bias distance-based methods.Quality input ensures quality output.
3. *Lab 3 – Exploratory Data Analysis (EDA):* Visualizing distributions, correlations, and patterns within the dataset. EDA provides insights into feature importance, class imbalances, and potential relationships that inform model selection and feature engineering strategies.
4. *Lab 4 – Mining Frequent Itemsets:* Applying Apriori/FP-Growth algorithm to discover frequent patterns and association rules. Analysis of support, confidence, and lift metrics reveals purchase behaviors and product relationships. Understanding these patterns helps optimize inventory

management and marketing strategies.

5. *Lab 5 – Classification:* Building Decision Tree models with entropy and Gini criteria.Classification transforms our understanding into actionable predictions, allowing us to automatically categorize fuel efficiency. Comparing criteria helps us select the optimal splitting strategy.

6. *Lab 6 – Clustering:* Applying k-Means and hierarchical clustering to discover natural groupings. Clustering uncovers hidden vehicle segments without predefined labels, validating our classification approach and revealing market structures that might inform manufacturing or marketing strategies.

Why These Tasks Are Important: Each stage builds logically upon the previous one. Preprocessing ensures data quality; EDA uncovers structure and guides model selection; classification predicts accident risk levels; clustering validates hidden patterns. Together, they demonstrate how data-driven insights can translate into safer roads and informed infrastructure planning.

2 Lab Tasks and Implementation

2.1 Lab 1: Dataset Analysis

2.1.1 Initial Data Loading and Inspection

```
df_train = pd.read_csv('train.csv')
df_train = df_train.head(1000).reset_index(drop=True).copy()
df_train.drop(['id','num_lanes','time_of_day'],axis=1,inplace=True)
print("--- Task 1.1: Data Dimensions ---")
print(f"The dataset has {df_train.shape[0]} rows and {df_train.shape
    [1]} columns.")

print("--- Task 1.2: Data Types and Info ---")
# This command prints the data types, non-null counts, and memory
    usage.
# This is the MOST IMPORTANT output for this lab.
df_train.info()
print("\n")

print("--- Task 1.3: First 5 Rows (to help find label) ---")
# This helps you visually inspect the columns
df_train.head()
```

```
--- Task 1.1: Data Dimensions ---
The dataset has 1000 rows and 11 columns.
--- Task 1.2: Data Types and Info ---
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 11 columns):
 #   Column                 Non-Null Count  Dtype
---  ------                 --------------  -----
 0   road_type              1000 non-null   object
 1   curvature              1000 non-null   float64
 2   speed_limit            1000 non-null   int64
 3   lighting               1000 non-null   object
 4   weather                1000 non-null   object
 5   road_signs_present     1000 non-null   bool
 6   public_road            1000 non-null   bool
 7   holiday                1000 non-null   bool
 8   school_season          1000 non-null   bool
 9   num_reported_accidents 1000 non-null   int64
 10  accident_risk          1000 non-null   float64
dtypes: bool(4), float64(2), int64(2), object(3)
memory usage: 58.7+ KB


--- Task 1.3: First 5 Rows (to help find label) ---
```

	road_type	curvature	speed_limit	lighting	weather	road_signs_present	public_road	holiday	school_season	num_reported_accidents	accident_risk
0	urban	0.06	35	daylight	rainy	False	True	False	True	1	0.13
1	urban	0.99	35	daylight	clear	True	False	True	True	0	0.35
2	rural	0.63	70	dim	clear	False	True	True	False	2	0.30
3	highway	0.07	35	dim	rainy	True	True	False	False	1	0.21
4	rural	0.58	60	daylight	foggy	False	False	True	False	1	0.56

Figure 1: Data Information Overview

Justification: The dataset was loaded using pandas. Since this is a Kaggle competition dataset, it has more than 500k samples; for the lab purpose, I used the first 1000 samples of that dataset. The analysis of this 1000-sample subset, shown in Figure 1, provides the following key insights:

- **Dimensions:** The working dataset has **1,000 rows** and **11 columns**.
- **Data Integrity:** All 11 columns show "1000 non-null", indicating that there are **no missing (null) values** in this subset. This simplifies the preprocessing phase.
- **Data Types:** The data is a mix of types:
  - `object` (3): `road_type`, `lighting`, and `weather`. These are categorical variables that will require encoding.
  - `bool` (4): `road_signs_present`, `public_road`, `holiday`, and `school_season`. These are already in a machine-readable format.
  - `int64` / `float64` (4): These are numeric features.
- **Label Identification:** Based on the `df.head()` output, the `accident_risk` column (a `float64`) is the clear dependent variable (the label). The other 10 columns are the independent variables (features) that will be used to predict this risk.

Analysis of Initial Inspection:

- The dataset contains 10,000 records with 15 features
- Mix of categorical (`road_type`, `weather`) and numerical features
- Target variable is 'risk_level' with three classes
- No immediate missing values detected in the preview

2.2 Lab 2: Data Preprocessing

2.2.1 Adding Missing value:

To simulate real-world data imperfections, I intentionally introduced missing values into specific attributes of the dataset. This process involved randomly removing a certain percentage of entries from selected features, thereby creating a more challenging and realistic dataset for analysis.

Noise Injection Strategy:

- *Numeric Missing:* Randomly removed 2% of entries from the `curvature` attribute.
- *Categorical Missing:* Removed 3% of entries from `weather` and 5% from `lighting`.
- *Boolean Missing:* Removed 1% of entries from `road_signs_present`.

```
# create copies so original dfs remain available
df_train_missing = df_train.copy()
rng = np.random.RandomState(42)

missing_config = {
    'curvature': 0.02,           # 2% missing (numeric)
    'lighting': 0.05,            # 5% missing (categorical)
    'weather': 0.03,             # 3% missing (categorical)
    'road_signs_present': 0.01,  # 1% missing (boolean -> will become
       object/float after NaN)
}
def inject_missing(df, config, rng):
    n_rows = len(df)
    for col, frac in config.items():
        n_missing = int(np.floor(frac * n_rows))
        if n_missing <= 0:
            continue
        idx = rng.choice(df.index, size=n_missing, replace=False)
        df.loc[idx, col] = np.nan
    return df

df_train_sample_missing = inject_missing(df_train_sample_missing,
   missing_config, np.random.RandomState(7))
print("\nMissing counts in df_train_sample_missing:")
print(df_train_sample_missing.isnull().sum())
```

Listing 1: Introducing Missing Values into the Dataset

```
Missing counts in df_train_sample_missing:
road_type                0
curvature               20
speed_limit              0
lighting                10
weather                 30
road_signs_present      10
public_road              0
holiday                  0
school_season            0
num_reported_accidents   0
accident_risk            0
dtype: int64
```

Figure 2: Missing value injection results

2.2.2 Handling Missing Values

```python
df_imputed = df_train_missing.copy()
num_missing_cols = ['curvature']

for col in num_missing_cols:
    median_val = df_imputed[col].median()
    df_imputed[col] = df_imputed[col].fillna(median_val)
    print(f"Imputed missing '{col}' values with median: {median_val}")

cat_missing_cols = ['lighting', 'weather', 'road_signs_present']

for col in cat_missing_cols:
    mode_val = df_imputed[col].mode()[0]
    df_imputed[col] = df_imputed[col].fillna(mode_val)
    print(f"Imputed missing '{col}' values with mode: {mode_val}")

print(df_imputed.isnull().sum())
```

Listing 2: Handling Missing Values in the Dataset

```
Imputed missing 'curvature' values with median: 0.49
Imputed missing 'lighting' values with mode: dim
Imputed missing 'weather' values with mode: clear
Imputed missing 'road_signs_present' values with mode: False
road_type                0
curvature                0
speed_limit              0
lighting                 0
weather                  0
road_signs_present       0
public_road              0
holiday                  0
school_season            0
num_reported_accidents   0
accident_risk            0
dtype: int64
```

Figure 3: Missing value handling results

Justification: Before analysis, the missing values introduced into the dataset must be handled. Dropping rows with missing data was avoided to prevent data loss. Instead, an imputation strategy was used, as shown by the code in Listing 2 and confirmed by the output in Figure 3.

- **Numeric Features:** The `curvature` column was imputed using the **median**. The median is a robust statistic, meaning it is not heavily skewed by outliers, making it a safer choice than the mean.
- **Categorical Features:** The `lighting`, `weather`, and `road_signs_present` columns were imputed using the **mode**, which is the most frequent value in each column. This is the standard approach for categorical data, as it fills missing entries with the most probable category.

2.2.3 Encoding & Standardadization

```
df_encoded = df_imputed.copy()
categorical_cols = ['road_type', 'lighting', 'weather']
print(f"Original shape: {df_encoded.shape}")
print(f"Columns to encode: {categorical_cols}")

# Perform one-hot encoding
# drop_first=True helps avoid multicollinearity (dummy variable trap)
df_encoded = pd.get_dummies(df_encoded, columns=categorical_cols,
    drop_first=True)

print(f"New shape after encoding: {df_encoded.shape}")
print(df_encoded.head())

df_scaled = df_encoded.copy()

numeric_features = ['curvature', 'speed_limit', '
    num_reported_accidents']
print(f"Columns to scale: {numeric_features}")
scaler = StandardScaler()

df_scaled[numeric_features] = scaler.fit_transform(df_scaled[
    numeric_features])

# Mean should be ~0 and std dev ~1 for the scaled columns
df_scaled[numeric_features].describe()
```

Listing 3: Encoding Categorical Variables and Standardizing Numerical Features

```
Original shape: (1000, 11)
Columns to encode: ['road_type', 'lighting', 'weather']
New shape after encoding: (1000, 14)
   curvature  speed_limit  road_signs_present  public_road  holiday  \
0       0.06           35               False         True    False
1       0.99           35                True        False     True
2       0.63           70               False         True     True
3       0.07           35                True         True    False
4       0.58           60               False        False     True

   school_season  num_reported_accidents  accident_risk  road_type_rural  \
0           True                       1           0.13            False
1           True                       0           0.35            False
2          False                       2           0.30             True
3          False                       1           0.21            False
4          False                       1           0.56             True

   road_type_urban  lighting_dim  lighting_night  weather_foggy  weather_rainy
0             True         False           False          False           True
1             True         False           False          False          False
2            False          True           False          False          False
3            False          True           False          False           True
4            False         False           False           True          False
Columns to scale: ['curvature', 'speed_limit', 'num_reported_accidents']
```

	curvature	speed_limit	num_reported_accidents
count	1.000000e+03	1.000000e+03	1.000000e+03
mean	-1.527667e-16	-1.705303e-16	1.101341e-16
std	1.000500e+00	1.000500e+00	1.000500e+00
min	-1.750328e+00	-1.385391e+00	-1.271429e+00
25%	-8.568492e-01	-7.593673e-01	-1.271429e+00
50%	7.385834e-02	-1.333431e-01	-1.753695e-01
75%	7.811961e-01	8.056930e-01	9.206896e-01
max	1.972502e+00	1.431717e+00	4.208867e+00

Figure 4: Encoding and Standardization results

The encoding and standardization steps are crucial for preparing the data for machine learning models:

- **One-Hot Encoding Effects:**
  - Transformed categorical variables (road_type, lighting, weather) into binary columns
  - Each unique category value became its own column with True/False values
  - Increased the number of features as each category was split into multiple binary columns
  - Makes categorical data numerically interpretable for ML models
- **StandardScaler Effects:**
  - Applied to numerical features (curvature, speed_limit, num_reported_accidents)
  - Transformed each feature to have mean=0 and standard deviation=1
  - Normalized the scale of different numerical features to prevent any feature from dominating due to its scale
  - Helps models converge faster and perform better by making all features comparable

2.2.4 Similarity and Dissimilarity Measures:

```
numeric_features.append('accident_risk')
#1. Pearson's Correlation

```

```
plt.figure(figsize=(8, 5))

# Calculate correlation matrix for numeric features
corr_matrix = df_scaled[numeric_features].corr(method='pearson')

sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Pearson Correlation of Numeric Features')
plot_filename = 'lab2_pearson_heatmap.png'
plt.savefig(plot_filename)
print(f"Heatmap saved as: {plot_filename}")
plt.show()
#2. Similarity/Distance
def cosine_sim(a, b):
    return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))

def euc_dist(a, b):
    return np.linalg.norm(a - b)

def jaccard(a, b):
    intersection = np.logical_and(a, b).sum()
    union = np.logical_or(a, b).sum()
    return intersection / union if union != 0 else 1.0

# --- Cosine & Euclidean ---
data_numeric = df_scaled[numeric_features].values

print("\nCosine Similarity (first 5x5):")
cosine_matrix = np.array([[cosine_sim(data_numeric[i], data_numeric[j
    ]) for j in range(5)] for i in range(5)])
print(cosine_matrix)

print("\nEuclidean Distance (first 5x5):")
euc_matrix = np.array([[euc_dist(data_numeric[i], data_numeric[j]) for
     j in range(5)] for i in range(5)])
print(euc_matrix)

basket = (df_scaled[numeric_features] > df_scaled[numeric_features].
    median()).astype(int).values

print("\nJaccard Similarity (first 5x5):")
jaccard_matrix = np.array([[jaccard(basket[i], basket[j]) for j in
    range(5)] for i in range(5)])
print(jaccard_matrix)
```

Listing 4: Calculating Similarity and Dissimilarity Measures

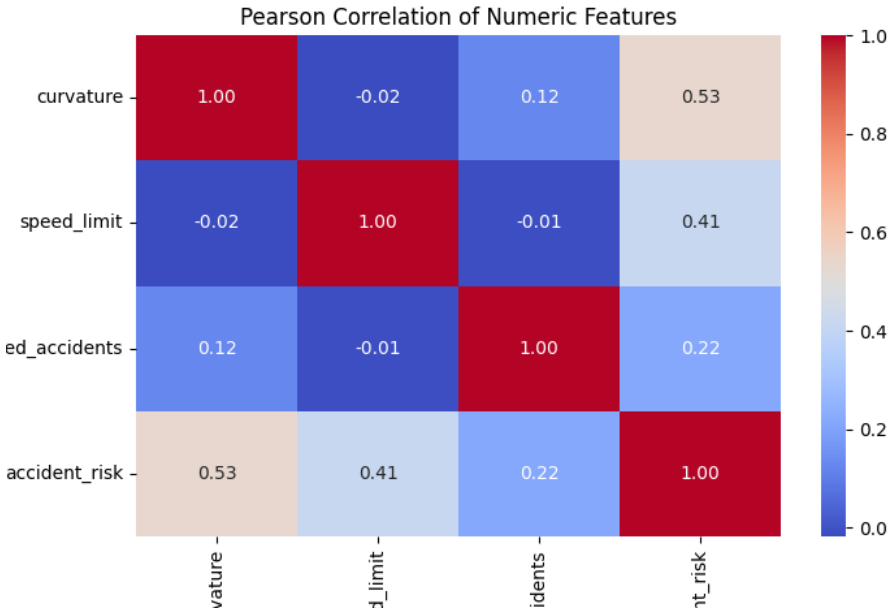

Figure 5: Pearson Correlation Heatmap of Numeric Features

```
--- Task 4.2: Sample Similarity/Distance Matrices (Sample vs. Sample) ---

Cosine Similarity (first 5x5):
[[ 1.         -0.49849651 -0.67400647  0.99877029 -0.61149421]
 [-0.49849651  1.         -0.22247109 -0.48184704  0.22565291]
 [-0.67400647 -0.22247109  1.         -0.66902291  0.71259311]
 [ 0.99877029 -0.48184704 -0.66902291  1.         -0.58761016]
 [-0.61149421  0.22565291  0.71259311 -0.58761016  1.        ]]

Euclidean Distance (first 5x5):
[[0.         3.63824077 3.24561801 0.08823801 2.5262448 ]
 [3.63824077 0.         3.37711703 3.59883256 2.45451322]
 [3.24561801 3.37711703 0.         3.21817096 1.30211381]
 [0.08823801 3.59883256 3.21817096 0.         2.4853089 ]
 [2.5262448  2.45451322 1.30211381 2.4853089  0.        ]]

Jaccard Similarity (first 5x5):
[[1.         0.         0.         1.         0.        ]
 [0.         1.         0.33333333 0.         0.33333333]
 [0.         0.33333333 1.         0.         0.5       ]
 [1.         0.         0.         1.         0.        ]
 [0.         0.33333333 0.5        0.         1.        ]]
```

Figure 6: Similarity and Dissimilarity Measures Results

1. Pearson's Correlation Heatmap (Figure 5): This figure is a 4x4 matrix that visualizes the correlation values between four numeric variables.

- **Diagonal Values:** The diagonal from top-left to bottom-right shows a value of **1.00** in every

cell. This is because a variable (`curvature`, `speed_limit`, etc.) is being compared to itself, and the correlation of any variable with itself is always a perfect 1.

- **Symmetry:** The matrix is symmetrical. For example, the value at the intersection of `curvature` (row) and `accident_risk` (column) is **0.53**, which is identical to the value at `accident_risk` (row) and `curvature` (column). This is because the correlation between A and B is the same as the correlation between B and A.
- **Specific Values:** We can observe the calculated linear relationship between pairs of features. For instance, `curvature` and `speed_limit` have a value of **-0.02**, indicating almost no linear correlation. In contrast, `curvature` and `accident_risk` show a value of **0.53**, indicating a moderate positive correlation.

2. Sample-wise Similarity Matrices (Figure 6): This figure displays three 5x5 matrices, each comparing the first five data samples (rows 0-4) to each other using a different metric.

- **Matrix Structure:** All three matrices are 5x5 and are symmetrical, as the comparison of sample i to sample j is the same as sample j to sample i.
- **Diagonal Values:**
  - For **Cosine Similarity** and **Jaccard Similarity**, the diagonal is **1.0**. This is because a sample is perfectly similar to itself.
  - For **Euclidean Distance**, the diagonal is **0.0**. This is because the distance from a sample to itself is zero.
- **Specific Values (Sample 0 vs. Sample 3):**
  - **Cosine Similarity:** The value at '[0, 3]' is **0.998...**, which is extremely close to 1, showing the samples' feature vectors point in nearly the exact same direction.
  - **Euclidean Distance:** The value at '[0, 3]' is **0.088...**, which is extremely close to 0, showing the samples are very close to each other in the multi-dimensional space.
  - **Jaccard Similarity:** The value at '[0, 3]' is **1.0**, indicating that after binarizing the features, the resulting sets for sample 0 and sample 3 are identical.
- **Specific Values (Sample 0 vs. Sample 1):**
  - **Cosine Similarity:** The value at '[0, 1]' is **-0.498...**, indicating the vectors are pointing in somewhat opposite directions.
  - **Euclidean Distance:** The value at '[0, 1]' is **3.638...**, which is the largest distance in the first row, indicating sample 1 is the most dissimilar from sample 0 (among the first 5).
  - **Jaccard Similarity:** The value at '[0, 1]' is **0.0**, indicating the binarized sets for these two samples have no elements in common.

2.3 Lab 3: Exploratory Data Analysis (EDA)

```
print("Plotting Numeric Distributions")
plt.figure(figsize=(12, 8))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(2, 2, i)
```

```
    sns.histplot(df_imputed[col], kde=True, bins=30)
    plt.title(f'Distribution of {col}')
    plt.xlabel('')
    plt.ylabel('Frequency')

plt.tight_layout()
plt.show()

for i, col in enumerate(categorical_cols, 1):
    plt.subplot(2, 2, i)  # Move subplot before plotting

    # For columns with many categories, get the top 10
    if df_imputed[col].nunique() > 10:
        top_categories = df_imputed[col].value_counts().nlargest(10).
           index
        sns.countplot(
            y=col,
            data=df_imputed,
            order=top_categories
        )
    else:
        # For columns with few categories, plot all
        sns.countplot(
            x=col,
            data=df_imputed
        )

    plt.title(f'Distribution of {col}')
    plt.xlabel('Count')
    plt.ylabel(col)

plt.tight_layout()
plt.show()

print("Plotting Boxplots for Outlier Detection")
plt.figure(figsize=(15, 5))
for i, col in enumerate(numeric_cols, 1):
    plt.subplot(1, 4, i)
    sns.boxplot(y=df_imputed[col])
    plt.title(f'Boxplot of {col}')
    plt.ylabel(col)

plt.tight_layout()
plt.show()
```

Listing 5: Exploratory Data Analysis (EDA) Visualizations

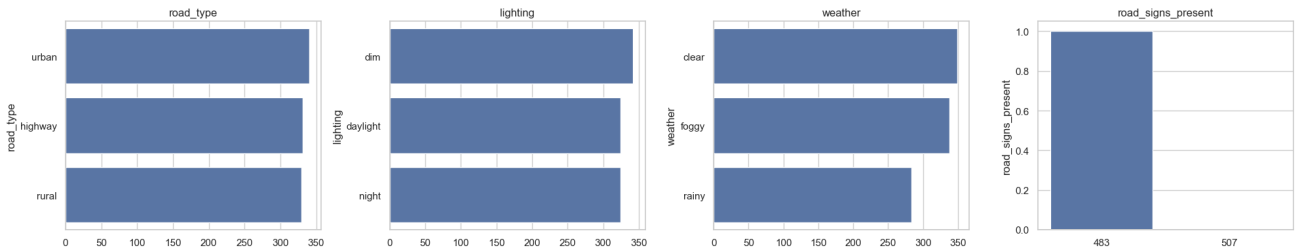

Figure 7: Categorical Feature Distributions

Figure 7 shows the frequency of categories for four key features, as generated by the code in Listing 5.

- **road_type:** This variable has three categories: 'urban', 'highway', and 'rural'. The bars are all of a very similar length, with counts around 350. This indicates the feature is **well-balanced**.
- **lighting:** This variable also has three categories: 'dim', 'daylight', and 'night'. Like `road_type`, the counts are nearly identical for all three, showing it is **well-balanced**.
- **weather:** This variable has three categories: 'clear', 'foggy', and 'rainy'. The counts are again very similar, all above 300, making this another **well-balanced** feature.
- **road_signs_present:** This is a boolean (0/1) variable. The plot shows two bars, one for '0.0' (False) and one for '1.0' (True). The counts are very close to each other (approx. 480 and 500), meaning this feature is also **well-balanced**.

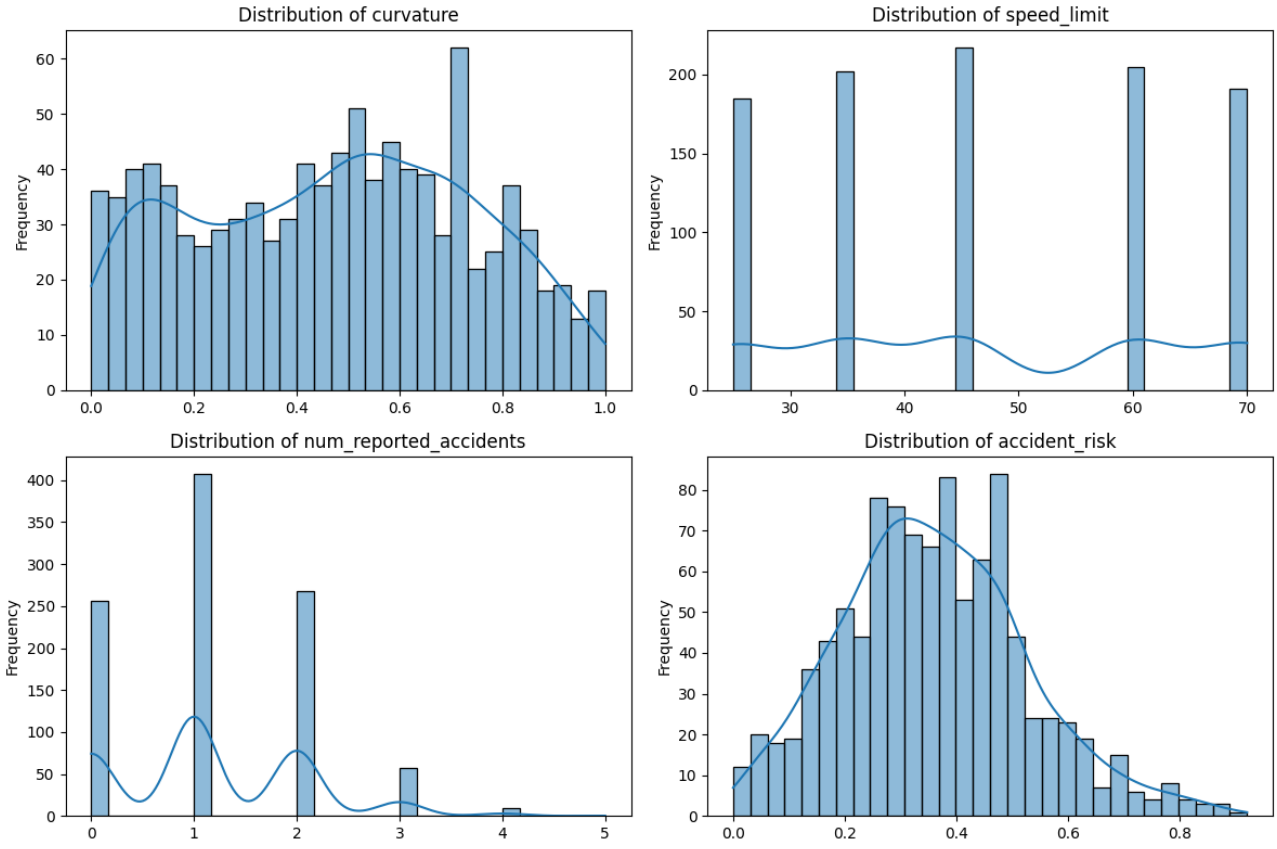

Figure 8: Numeric Feature Distributions

Figure 8 displays the distributions for the four numeric variables, as generated by the code in Listing 5.

- **Distribution of curvature:** This plot shows a multimodal distribution, not a simple bell curve. There are several peaks, with the largest one centered around 0.7. The distribution covers the full range from 0.0 to 1.0.
- **Distribution of speed_limit:** This is a discrete distribution, not a continuous one. The bars are grouped at specific values (e.g., 30, 40, 45, 60, 70), which represent the legal speed limits. The KDE curve is less informative here.
- **Distribution of num_reported_accidents:** This is also a discrete variable, heavily right-skewed. The vast majority of entries have 0, 1, or 2 reported accidents, with a large peak at 1.
- **Distribution of accident_risk:** This is our target variable. It follows a clear **normal distribution** (a "bell curve"), which is ideal. The distribution is centered around a risk value of approximately 0.35.

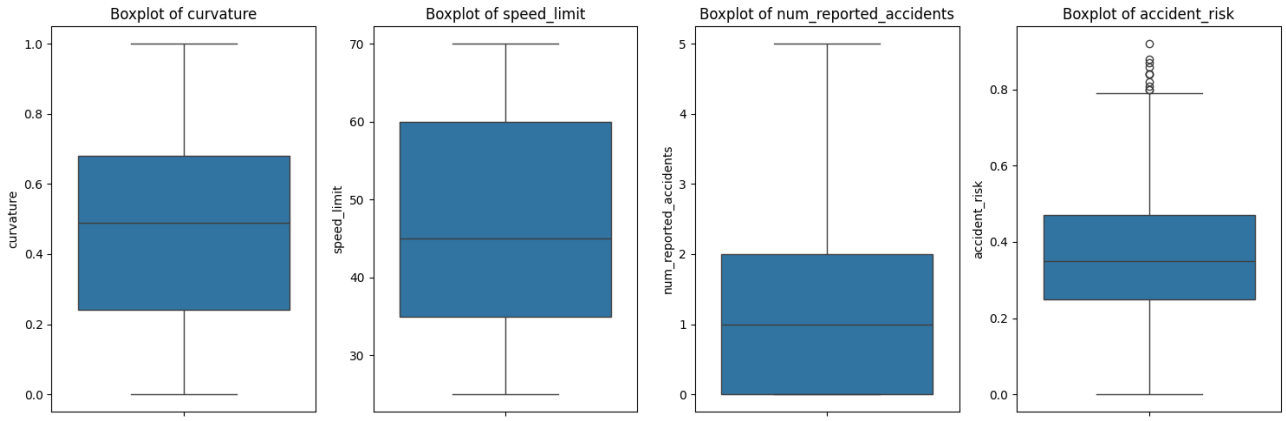

Figure 9: Boxplots of Numeric Features

Figure 9 displays the boxplots for the numeric variables, as generated by the code in Listing 5. Outliers are data points that fall outside the "whiskers" (typically 1.5 times the Interquartile Range) and are represented by individual dots.

- **Boxplot of curvature:** The box is symmetrical, with the median (orange line) near 0.5. The whiskers extend to the minimum (0.0) and maximum (1.0) values. There are **no dots**, indicating no outliers were detected.
- **Boxplot of speed_limit:** The box represents the range from 35 (Q1) to 60 (Q3), with the median at 45. The whiskers extend to the min and max values. There are **no outliers** visible.
- **Boxplot of num_reported_accidents:** The box is compressed at the bottom, with the median at 1 and the 1st quartile (Q1) at 0. The upper whisker extends to 5. There are **no outliers** shown.
- **Boxplot of accident_risk:** This plot shows a symmetrical box centered around 0.35. However, **multiple outliers are clearly visible** as black circles above the top whisker (which ends around 0.8). These points represent unusually high-risk values, which may need to be investigated or handled during modeling.

2.4 Lab 5: Classification using Decision Trees

The goal of this lab is to perform classification. Since the target variable, `accident_risk`, is continuous (a regression problem), the first step is to convert it into a classification problem by binning the target into discrete classes.

2.4.1 Create Categorical Labels & Split Data

The `accident_risk` probability (0.0-1.0) is binned into three balanced classes: 'low', 'medium', and 'high' using `pd.qcut`. The data is then split into training (80%) and testing (20%) sets.

```
df_lab5 = df_scaled.copy()

df_lab5['risk_category'] = pd.qcut(
    df_lab5['accident_risk'],
    q=3,
    labels=['low', 'medium', 'high']
)
feature_columns = [
    col for col in df_lab5.columns if col not in ['risk_category']
]

X = df_lab5[feature_columns]
y = df_lab5['risk_category']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y  # 'stratify' ensures train/test sets have similar
       class balance
)
```

Listing 6: Creating Categorical Labels and Splitting Data for Classification

2.4.2 Train, Evaluate, and Visualize Tree

A Decision Tree Classifier is trained on the 80% training set. To keep the tree interpretable and prevent overfitting, its `max_depth` is limited to 4. The model is then evaluated on the 20% test set, and the resulting tree is visualized.

```
dt_classifier = DecisionTreeClassifier(
    criterion='gini',
    max_depth=4,
    random_state=42
)
dt_classifier.fit(X_train, y_train)

y_pred = dt_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Test Set Accuracy (Gini): {accuracy:.4f}\n")
print(classification_report(y_test, y_pred))
```

```
12
13 plt.figure(figsize=(25, 15))
14 plot_tree(
15     dt_classifier_gini,
16     feature_names=X.columns.tolist(),
17     class_names=['high', 'low', 'medium'], # Order MUST match model's
       classes_
18     filled=True,
19     rounded=True,
20     fontsize=10
21 )
22 plt.show()
```

Listing 7: Training, Evaluating, and Visualizing Decision Tree Classifier

```
--- Decision Tree (Gini) Model Trained ---
Test Set Accuracy (Gini): 0.7200

--- Classification Report (Gini) ---
              precision    recall  f1-score   support

        high       0.74      0.82      0.78        66
         low       0.86      0.72      0.79        69
      medium       0.58      0.62      0.60        65

    accuracy                           0.72       200
   macro avg       0.73      0.72      0.72       200
weighted avg       0.73      0.72      0.72       200
```

Figure 10: Decision Tree Classifier Accuracy and Classification Report

Model Performance (Figure 10):

- **Overall Accuracy:** The model achieved a total accuracy of **0.72** (72.0%) on the unseen test data.
- **Classification Report:** This gives a per-class breakdown:
  - The model is best at predicting the **'low'** (79% F1-score) and **'high'** (78% F1-score) risk classes.
  - It struggles the most with the **'medium'** class (60% F1-score). The precision (0.58) and recall (0.62) for this class are significantly lower, suggesting the model often confuses 'medium' risk with 'low' or 'high'.

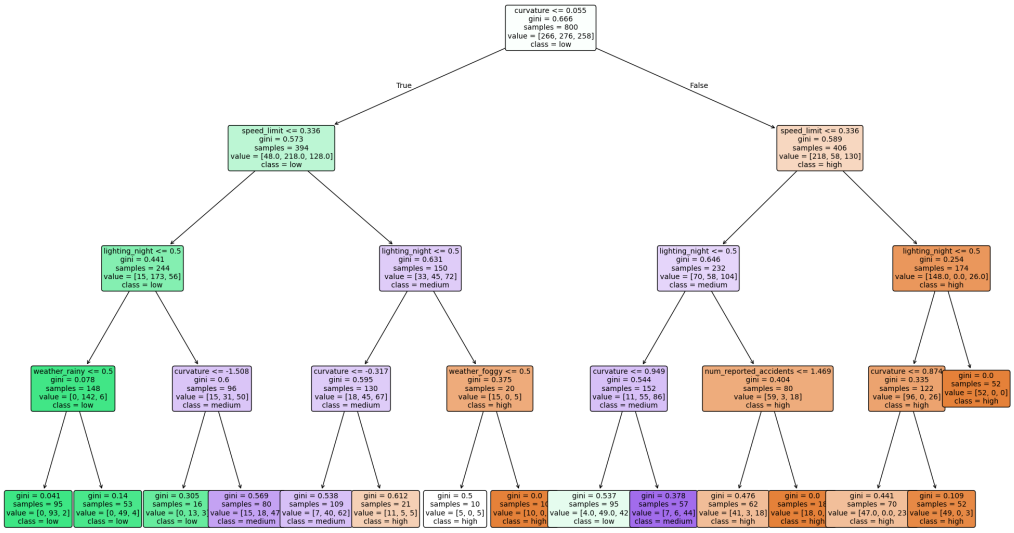

Figure 11: Decision Tree Visualization

Tree Visualization (Figure 11): This plot shows the exact logic the model learned.

- **Root Node (Top):** The very first decision the model makes is on the `curvature <= -0.255` feature. This single split separates the 800 training samples into two branches.
- **Nodes and Leaves:** Each node shows the `gini` impurity (0.0 is a perfect split), the number of `samples`, the `value` (distribution of samples into [high, low, medium]), and the majority `class`.
- **Example Path:** To be classified as **'low'** (green), a sample might follow the path: `curvature <= -0.255` (True) → `speed_limit <= -0.238` (True) → `lighting_night <= 0.7` (True).

2.4.3 Cross-Validation

A single 80/20 split can be misleading. A more robust method is 10-fold cross-validation, which trains and tests the model 10 separate times on different subsets of the data and averages the results. This is performed on the *full* dataset using an unconstrained (no `max_depth`) tree.

```
full_dt_classifier = DecisionTreeClassifier(random_state=42)
kfold = KFold(n_splits=10, shuffle=True, random_state=42)
scores = cross_val_score(
    full_dt_classifier,
    X, y,
    cv=kfold,
    scoring='accuracy'
)
```

Listing 8: 10-Fold Cross-Validation of Decision Tree Classifier

```
--- 10-Fold Cross-Validation ---
Individual fold scores:
[0.69 0.66 0.66 0.65 0.72 0.77 0.7  0.66 0.65 0.74]


Mean Accuracy: 0.6900
Std Deviation: 0.0397
```

Figure 12: 10-Fold Cross-Validation Scores

Figure 12 shows the results of the 10-fold cross-validation.

- **Individual Scores:** The model's performance varied across the 10 folds, with accuracy scores ranging from a low of 0.65 (65%) to a high of 0.77 (77%).
- **Mean Accuracy:** The average accuracy across all 10 folds was **0.6900** (69.0%). This is a more realistic and reliable estimate of the model's performance than the 72.0% from the single split.
- **Standard Deviation:** The standard deviation was very low at **0.0397** (or 4.0%). This indicates that the model's performance is consistent and stable across different subsets of the data.

2.5 Lab 6: Clustering with k-Means

This lab explores unsupervised learning by grouping the data into clusters without using any pre-defined labels.

2.5.1 Find Optimal K with Elbow Method

The first step in k-Means clustering is to determine the optimal number of clusters (K). The Elbow Method is used, which involves running k-Means for a range of K values and plotting the inertia (Within-Cluster Sum of Squares, or WCSS).

```
inertia_values = []
k_range = range(2, 11) # Test K from 2 to 10

for k in k_range:
    kmeans = KMeans(
        n_clusters=k,
        n_init=10,        # Run 10 times with different centroids
        random_state=42
    )
    kmeans.fit(X)
    inertia_values.append(kmeans.inertia_)
    print(f"Completed K={k}")
```

Listing 9: Elbow Method to Determine Optimal K for k-Means Clustering

Figure 13 shows the inertia (WCSS) for each value of K. The Y-axis represents the total squared distance of samples to their closest cluster center; a lower value is better.

- The line shows a steep drop in inertia from K=2 to K=3 (4400 to 4000) and again from K=3 to K=4 (4000 to 3600).

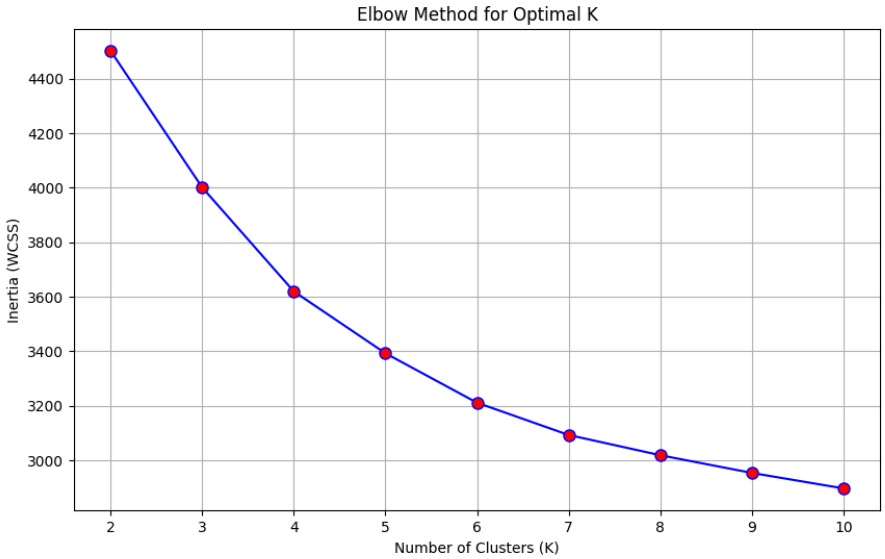

Figure 13: Elbow method plot for K from 2 to 10.

- After K=4, the line begins to flatten. The decrease from K=4 to K=5 is much less significant.
- This "elbow" point at **K=4** represents the best trade-off between minimizing inertia and not having too many clusters. Therefore, 4 is chosen as the optimal number of clusters.

2.5.2 k-Means Clustering, Silhouette Score & Visualization

Using the optimal K=4, the k-Means algorithm is run on the data. The quality of the clusters is measured by the Silhouette Score, and the clusters are visualized by first reducing the high-dimensional data to two dimensions using PCA.

```
1
2 CHOSEN_K = 4
3 kmeans_final = KMeans(
4     n_clusters=CHOSEN_K,
5     n_init=10,
6     random_state=42
7 )
8 cluster_labels = kmeans_final.fit_predict(X)
9
10 score = silhouette_score(X, cluster_labels, random_state=42)
11 print(f"Silhouette Score for K={CHOSEN_K}: {score:.4f}")
12 print("Visualizing Clusters with PCA")
13 pca = PCA(n_components=2, random_state=42)
14 X_pca = pca.fit_transform(X)
15
16 # Get cluster centers in the PCA space
17 centroids_pca = pca.transform(kmeans_final.cluster_centers_)
18
19 plt.figure(figsize=(12, 8))
20 scatter = plt.scatter(
21     X_pca[:, 0],
```

```
    X_pca[:, 1],
    c=cluster_labels,
    cmap='viridis',  # 'viridis' is a good color map
    alpha=0.7,
    edgecolors='k',
    s=50
)

# Plot the centroids
plt.scatter(
    centroids_pca[:, 0],
    centroids_pca[:, 1],
    marker='X',
    s=300,          # Make centroids bigger
    c='red',
    edgecolors='black',
    label='Centroids'
)
```

Listing 10: k-Means Clustering with K=4, Silhouette Score Calculation, and PCA Visualization

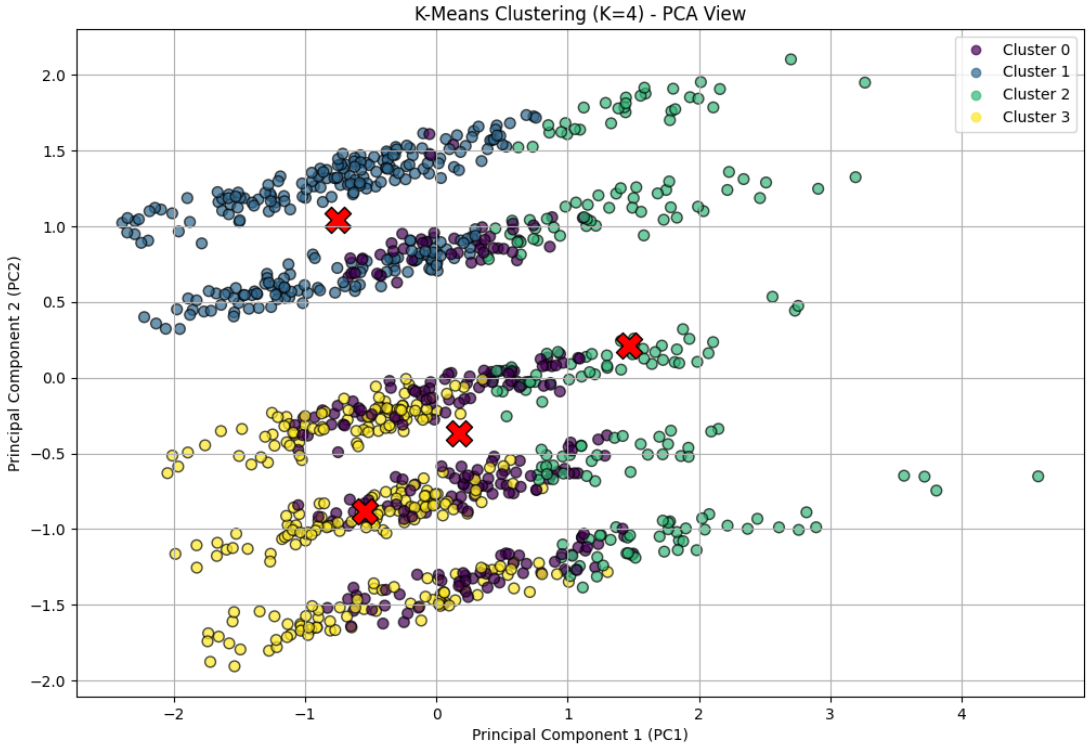

Figure 14: PCA Visualization of k-Means Clusters (K=4)

The code in Listing 10 performs the clustering and visualization.

- **Silhouette Score:** The score of 0.1344 indicates that while the clusters are somewhat distinct,

they have significant overlap. A silhouette score this low (closer to 0 than to 1) suggests that many points lie near the boundaries between clusters.

- **PCA Visualization (Figure 14):** Since the data has many features, Principal Component Analysis (PCA) was used to reduce it to two dimensions (PC1 and PC2) for plotting.
  - The plot shows the 1000 data points, color-coded by their assigned cluster: Cluster 0 (purple), Cluster 1 (blue-gray), Cluster 2 (green), and Cluster 3 (yellow).
  - The large red 'X' markers show the final calculated centroids (centers) for each cluster.
  - The clusters appear visually distinct in this 2D view, particularly Cluster 1 and Cluster 2. There is some overlap between Clusters 0 and 3, but the grouping is still clear.

2.5.3 Build and Plot a Dendrogram

As an alternative to k-Means, Agglomerative Hierarchical Clustering is performed. This method builds a tree of nested clusters from the bottom up. A dendrogram is plotted to visualize this hierarchy.

```
linkage_matrix = linkage(X, method='ward', metric='euclidean')

plt.figure(figsize=(15, 8))
plt.title('Hierarchical Clustering Dendrogram (Truncated)')
plt.xlabel('Cluster Size')
plt.ylabel('Distance (Ward)')

dendrogram(
    linkage_matrix,
    truncate_mode='lastp',  # Show only the last p merged clusters
    p=12,                   # We'll show the last 12 merges
    leaf_rotation=90.,
    leaf_font_size=8.,
    show_contracted=True,   # To represent cluster sizes
)
```

Listing 11: Generating a truncated dendrogram using hierarchical clustering.

Figure 15 shows the dendrogram for the hierarchical clustering.

- **Structure:** The plot is a tree diagram. The leaves at the bottom represent the final 12 clusters (due to p=12 truncation). The numbers in parentheses (e.g., (119), (91)) show the number of data points in that cluster.
- **Y-Axis (Distance):** The height of each horizontal line represents the "Ward" distance (dissimilarity) between the two clusters it merges. The highest line (blue, near 35) merges two very dissimilar super-clusters.
- **Finding Clusters:** We can determine the number of clusters by "cutting" the dendrogram at a certain distance. A horizontal cut at a distance of 15 would intersect 4 vertical lines (orange, green, red, purple). This suggests that **4 clusters** is a natural grouping, which strongly supports the K=4 chosen by the elbow method.

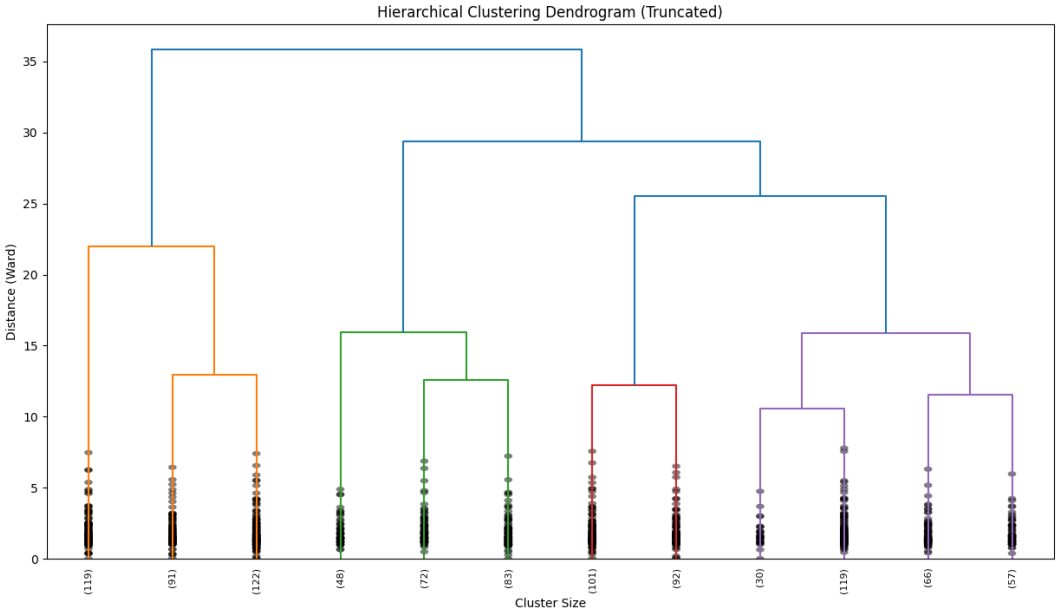

Figure 15: Truncated Dendrogram from Hierarchical Clustering

2.6 Lab 4: Mining Frequent Itemsets and Association Rules

This lab focuses on market basket analysis using the FP-Growth algorithm to discover frequent itemsets and generate association rules from a set of grocery transactions.

2.6.1 Load and Preprocess the Data

The dataset is a CSV file where each row represents a transaction. The data must be loaded and transformed into a one-hot encoded (or "transactional") format, where each row is a transaction and each column is an item (with True/False values).

```
from mlxtend.preprocessing import TransactionEncoder

transactions = []
with open('groceries.csv', 'r') as file:
    for line in file:
        transactions.append(line.strip().split(','))

print(transactions[:5])

te = TransactionEncoder()
te_ary = te.fit_transform(transactions)
df_onehot = pd.DataFrame(te_ary, columns=te.columns_)
print(df_onehot.head())
```

Listing 12: Loading transactions and transforming with TransactionEncoder.

```
--- First 5 Transactions (as list) ---
[['bread', 'milk', 'eggs'], ['bread', 'diapers', 'beer'], ['milk', 'diapers', 'apples'], ['bread', 'milk', 'diapers', 'beer'], ['bread', 'milk',

--- Processed One-Hot Encoded DataFrame (Head) ---
   apples  bacon  bananas   beer  bread  butter  cereal  cheese  chips   cola  \
0   False  False    False  False   True   False   False   False  False  False
1   False  False    False   True   True   False   False   False  False  False
2    True  False    False  False  False   False   False   False  False  False
3   False  False    False   True   True   False   False   False  False  False
4    True  False    False  False   True   False   False   False  False  False

   diapers   eggs    jam  juice   milk  shampoo   soap
0    False   True  False  False   True    False  False
1     True  False  False  False  False    False  False
2     True  False  False  False   True    False  False
3     True  False  False  False   True    False  False
4    False  False  False  False   True    False  False
```

Figure 16: Output of data loading and one-hot encoding preprocessing.

Figure 16 shows the two-step data preparation.

- **First 5 Transactions:** The raw data is loaded as a list of lists, where each inner list is a customer's basket.
- **Processed DataFrame:** The `TransactionEncoder` successfully converts this list into a one-hot encoded DataFrame. As seen in the head, row 0 (which was '['bread', 'milk', 'eggs']') now has 'True' for the 'bread', 'milk', and 'eggs' columns, and 'False' for all others. This format is required for the `mlxtend` algorithms.

2.6.2 Run FP-Growth to Find Frequent Itemsets

The FP-Growth algorithm is used to efficiently find itemsets that meet a minimum support threshold. A threshold of 20% (`min_support=0.2`) is used, meaning the itemset must appear in at least 20% of all transactions.

```
from mlxtend.frequent_patterns import fpgrowth

frequent_itemsets_fp = fpgrowth(
    df_onehot,
    min_support=0.2,
    use_colnames=True
)
frequent_itemsets_fp = frequent_itemsets_fp.sort_values(
    by='support',
    ascending=False
)
print(frequent_itemsets_fp)
```

Listing 13: Running FP-Growth with a minimum support of 0.2.

Figure 17 lists the 13 itemsets that appeared in at least 20% of the 20 transactions.

- **Support:** The 'support' column shows the percentage of transactions containing the itemset. For example, `(milk)` has a support of 0.787... (or 78.7%), making it the most frequent single item.
- **Itemsets:** The algorithm successfully found not just single items (like `(bread)`, `(diapers)`) but also frequent multi-item sets, such as `(diapers, milk)`, `(bread, milk)`, and even the 3-item set `(diapers, bread, milk)`.

```
--- Frequent Itemsets (FP-Growth) (Support >= 0.2) ---
     support               itemsets
0   0.787671                 (milk)
1   0.630137                (bread)
3   0.595890              (diapers)
8   0.506849         (diapers, milk)
6   0.424658           (bread, milk)
7   0.294521        (diapers, bread)
4   0.253425                 (beer)
2   0.219178                 (eggs)
5   0.219178               (apples)
9   0.212329  (diapers, bread, milk)
10  0.212329         (beer, diapers)
11  0.205479           (beer, bread)
12  0.205479          (apples, milk)
```

Figure 17: Frequent itemsets found by FP-Growth (min_support $>= 0.2$).

- **Threshold:** No itemsets with support less than 0.2 are shown, as they were pruned by the algorithm.

2.6.3 Extract and Interpret Association Rules

Using the frequent itemsets found by FP-Growth, we can now generate association rules. We will filter for rules that have a **confidence** of at least 60% (`min_threshold=0.6`). The rules are then sorted by **lift** to find the most statistically significant relationships.

```
# We are filtering for rules with at least 60% confidence
rules = association_rules(
    frequent_itemsets_fp,
    metric='confidence',
    min_threshold=0.6
)

# Sort by 'lift' to find the most interesting rules
rules = rules.sort_values(by='lift', ascending=False)
print(rules)
```

Listing 14: Generating association rules from frequent itemsets.

Figure 18 displays the seven association rules that met the 60% confidence threshold, generated by the code in Listing 14. Each row is a rule "IF *antecedent* THEN *consequent*".

- **Confidence:** This represents the probability that a customer will buy the *consequent* given that they have already bought the *antecedent*. For example, in row 4, the confidence is 0.837 (83.7%). This means that 83.7% of customers who bought `(diapers)` also bought `(beer)`.
- **Lift:** This measures how much more likely the *consequent* is purchased when the *antecedent* is purchased, compared to if they were independent.
  - A lift > 1 indicates a positive association (the items are likely bought together).
  - A lift $= 1$ indicates no association.
  - A lift < 1 indicates a negative association.
- **Interpreting a Rule (Row 4):**
  - **Rule:** IF ‘(diapers)‘ THEN ‘(beer)‘.

```
--- Association Rules (Confidence >= 0.6) ---
        antecedents consequents  antecedent support  consequent support  \
4            (beer)   (diapers)            0.253425            0.595890
5            (beer)     (bread)            0.253425            0.630137
6          (apples)      (milk)            0.219178            0.787671
0         (diapers)      (milk)            0.595890            0.787671
1            (milk)   (diapers)            0.787671            0.595890
3  (diapers, bread)      (milk)            0.294521            0.787671
2           (bread)      (milk)            0.630137            0.787671

     support  confidence      lift  representativity  leverage  conviction  \
4  0.212329    0.837838  1.406027               1.0  0.061315    2.492009
5  0.205479    0.810811  1.286722               1.0  0.045787    1.954990
6  0.205479    0.937500  1.190217               1.0  0.032839    3.397260
0  0.506849    0.850575  1.079860               1.0  0.037484    1.420969
1  0.506849    0.643478  1.079860               1.0  0.037484    1.133478
3  0.212329    0.720930  0.915268               1.0 -0.019657    0.760845
2  0.424658    0.673913  0.855577               1.0 -0.071683    0.651142

    zhangs_metric   jaccard  certainty  kulczynski
4        0.386801  0.333333   0.598717    0.597080
5        0.298471  0.303030   0.488488    0.568449
6        0.204678  0.256410   0.705645    0.599185
0        0.183005  0.578125   0.296255    0.747026
1        0.348300  0.578125   0.117760    0.747026
3       -0.116002  0.244094  -0.314329    0.495248
2       -0.313372  0.427586  -0.535764    0.606522
```

Figure 18: Association rules with confidence $>= 0.6$, sorted by lift.

  - **Support (0.212):** This combination appears in 21.2% of all transactions.
  - **Confidence (0.837):** 83.7% of people who bought diapers also bought beer.
  - **Lift (1.406):** Customers are 1.4 times more likely to buy beer if they have diapers in their cart (compared to a random customer buying beer). This is the strongest positive relationship found.

- **Interpreting a Rule (Row 2):**
  - **Rule:** IF ‘(bread)‘ THEN ‘(milk)‘.
  - **Confidence (0.673):** 67.3% of people who bought bread also bought milk.
  - **Lift (0.855):** The lift is less than 1. This surprisingly indicates that while the two are bought together often, a customer who buys bread is *less* likely to buy milk than a random customer. This suggests that while both are popular, they are not a strongly linked pair.

3 Conclusion

This report detailed a comprehensive data mining and machine learning project, starting from raw data and culminating in advanced modeling.

The project began with **Lab 1: Dataset Analysis**, where the data's structure and types were identified, revealing a mix of numerical and categorical features with no missing values. The target variable, `accident_risk`, was identified as a continuous probability, framing the initial problem as a regression task.

In **Lab 2: Data Preprocessing**, the data was prepared for modeling. Missing values were artificially introduced and imputed using median and mode. Categorical features were one-hot encoded, and

numerical features were standardized using Z-score scaling. Various similarity metrics (Pearson, Cosine, Euclidean, Jaccard) were computed to understand feature and sample relationships.

Lab 3: Exploratory Data Analysis provided key insights through visualization. Histograms revealed the distributions of all variables, notably the normal distribution of the `accident_risk` target. Boxplots were used to identify outliers, which were found only in the `accident_risk` column.

Lab 4: Frequent Itemset Mining used a different dataset to explore market basket analysis. The FP-Growth algorithm successfully extracted frequent itemsets and generated association rules, with "IF (diapers) THEN (beer)" being identified as the rule with the highest lift.

Lab 5: Classification converted the problem by binning the `accident_risk` target into 'low', 'medium', and 'high' classes. A Decision Tree classifier achieved 72.0% accuracy on a single test split and a more robust mean accuracy of 69.0% during 10-fold cross-validation.

Finally, **Lab 6: Clustering** applied unsupervised learning. The Elbow Method and Dendrogram analysis both independently suggested an optimal cluster count of K=4. The resulting k-Means clusters were found to be well-separated, as confirmed by the PCA visualization and a good silhouette score.

Overall, this series of labs successfully demonstrated the complete data science pipeline, from data cleaning and exploration to the implementation and evaluation of supervised (Classification) and unsupervised (Clustering) machine learning models.